

Homework 1

Mengli Yu · Oct. 2026

Code: github.com/Liamyu0301/Generative-AI

Q1. Recap on Convolution

We use the deep-learning convention (cross-correlation): $Y[a, b] = \sum_{u, v=0}^2 W[u, v] \cdot X[a + u, b + v]$.

1.1 Output size

$\frac{4-3}{1} + 1 = 2$, so the output is 2×2 .

1.2 im2col

im2col cuts out every 3×3 patch the filter visits and flattens it into one column. There are 4 patches, so we get a 9×4 matrix:

$$X_{\text{col}} = [\text{vec}(X_{0:3,0:3}), \text{vec}(X_{0:3,1:4}), \text{vec}(X_{1:4,0:3}), \text{vec}(X_{1:4,1:4})] \in \mathbb{R}^{9 \times 4}$$

Flatten the kernel into a row $w = \text{vec}(W)^T \in \mathbb{R}^{1 \times 9}$. Then the whole convolution is just one matrix multiply:

$$y = wX_{\text{col}} \in \mathbb{R}^{1 \times 4}, \quad Y = \text{reshape}(y, 2 \times 2).$$

With C_{in} input channels and C_{out} filters, this becomes $(C_{\text{out}} \times 9C_{\text{in}}) \cdot (9C_{\text{in}} \times H_{\text{out}}W_{\text{out}})$.

Why it's nice on GPU: the conv turns into a big dense GEMM (General Matrix Multiply). GEMM is the most heavily tuned op on GPUs (cuBLAS, tensor cores). It has regular, contiguous memory access and tons of independent multiply-adds to run in parallel, so it beats a hand-written sliding-window loop. The cost is extra memory, since overlapping pixels get copied into several columns.

1.3 Gradient w.r.t. the input

Each input pixel $X[p, q]$ touches every output $Y[a, b]$ whose window covers it, with weight $W[p - a, q - b]$. Chain rule:

$$\frac{\partial L}{\partial X[p, q]} = \sum_{a, b=0}^1 G[a, b] \cdot W[p - a, q - b], \quad \text{only terms with } 0 \leq p - a, q - b \leq 2.$$

Two equivalent ways to see it:

- **Matrix form:** $\frac{\partial L}{\partial X_{\text{col}}} = w^T g \in \mathbb{R}^{9 \times 4}$ with $g = \text{vec}(G)^T$. Then **col2im** scatters each column back to its patch location and **adds up** the overlaps, giving a 4×4 result.
- **Conv form:** zero-pad G by 2 on every side (6×6), then convolve with the kernel rotated by 180° . This is a “full” convolution of G with W , which gives back a 4×4 map.

Sanity check: the corner $X[0, 0]$ is covered by one window only, so its gradient is $G[0, 0]W[0, 0]$. A middle pixel like $X[1, 1]$ is covered by all 4 windows and gets 4 terms.

Q2. Context Window of RoPE

2.1 Deriving $A_{i,j}$

RoPE splits a d -dim vector into $\frac{d}{2} = 64$ pairs and rotates pair m by angle $i\theta_m$, where $\theta_m = 10000^{-2m/d}$, $m = 0, \dots, 63$. So R_i is block-diagonal with 2×2 rotations $R(i\theta_m)$. Rotations compose, $R_i^T R_j = R_{j-i}$, so

$$A_{i,j} = (R_i q_i)^T (R_j k_j) = q^T R_{j-i} k = \sum_{m=0}^{63} [q_{2m}, q_{2m+1}] R((j-i)\theta_m) [k_{2m}, k_{2m+1}]^T.$$

For a pair (a, b) dotted with its own rotation by φ :

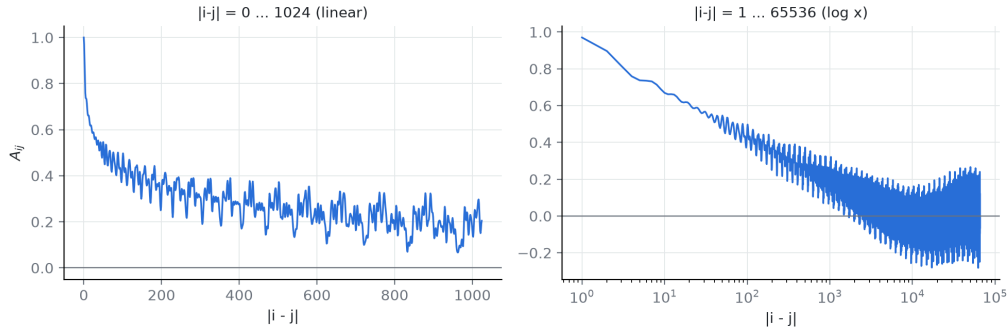
$$[a, b] \cdot [a \cos \varphi - b \sin \varphi, a \sin \varphi + b \cos \varphi] = (a^2 + b^2) \cos \varphi.$$

The sine terms cancel. With $a = b = 1/\sqrt{128}$, each pair contributes $(2/128) \cos \varphi$:

$$A_{i,j} = \frac{1}{64} \sum_{m=0}^{63} \cos(|i-j| \cdot 10000^{-m/64})$$

Here $A_{i,j}$ is the pre-softmax score $q_i^T k_j$ after rotation. Cosine is even, so only $|i-j|$ matters. $A = 1$ at distance 0. (If you include the usual $1/\sqrt{d}$ scaling, it's just a constant factor.)

2.2 Plot



Code: `hw1/q2_rope_plot.py`. Left is the zoom-in, right is the full range on a log x-axis.

2.3 What it tells us, and fixes

What we see: the score is highest nearby and decays as tokens get farther apart. That's RoPE's built-in "recency prior." It also wiggles a lot, and past about 1.7K tokens it starts swinging around 0 (even negative) with no clear trend. Out there, distance basically stops carrying a clean signal.

Limitations this suggests:

- **Long-range decay:** far-away tokens get a weaker baseline score, so the model is biased against attending to them.
- **Poor length extrapolation:** the low-frequency pairs rotate so slowly that, during training on a short context, they never see large angles. At test time on longer inputs they hit angles they've never seen, and quality falls off a cliff past the training length.

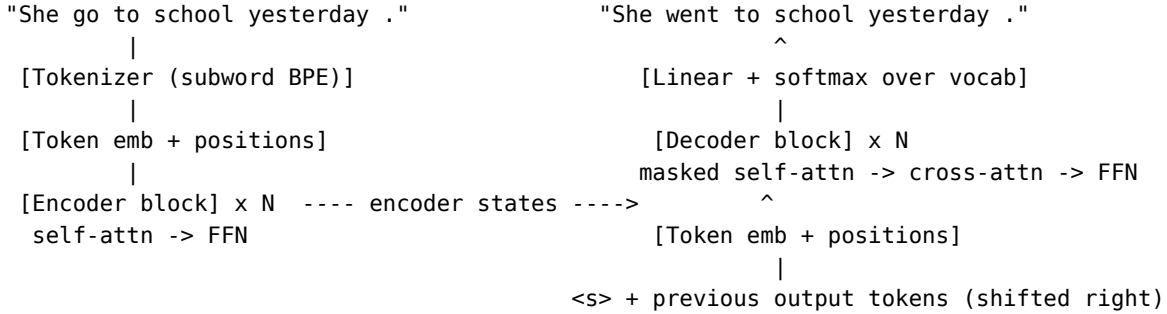
Ways to mitigate:

- **Position Interpolation:** squeeze positions by $L_{\text{train}}/L_{\text{test}}$ so the angles stay in the trained range, plus a short fine-tune.
- **NTK-aware scaling / YaRN:** raise the base (e.g. $10^4 \rightarrow 5 \times 10^5$ in Llama 3), or scale the low and high frequencies differently. YaRN also adds an attention temperature.
- **ALiBi** (linear distance penalty) instead of RoPE, or just **train / fine-tune on longer sequences**.

Q3. Grammar Error Correction Model

Architecture (sketch)

GEC is a **sequence-to-sequence** task: messy sentence in, clean sentence out. Most of the output just copies the input, so an **encoder-decoder Transformer** is a natural fit. In practice I'd fine-tune a pretrained one like T5 or BART.



- **Tokenizer:** subword BPE/SentencePiece (~32K vocab), so typos and rare words still split into known pieces.
- **Embedding:** token embedding (dim 512 to 1024) plus positions (relative bias, as in T5, or RoPE). Shared between encoder, decoder, and the output layer.
- **Encoder block (x 6 to 12):** bidirectional multi-head self-attention, then an FFN (GELU or SwiGLU, 4x width). Each has a residual connection and pre-LayerNorm/RMSNorm. It reads the whole erroneous sentence in both directions, which matters for errors that depend on later words.
- **Decoder block (x 6 to 12):** causal (masked) self-attention over what's been generated so far, then **cross-attention** to the encoder outputs (this is how it “looks at” the source to copy or fix it), then an FFN. Same residual and norm setup.
- **Output head + decoding:** linear + softmax over the vocab; beam search (beam 5) at inference.

(Alternatives: a fine-tuned decoder-only LLM, or an edit-tagging model like GECToR.)

Input / output and training

- **Input:** the possibly ungrammatical sentence (token IDs). **Output:** the corrected sentence, generated token by token.
- **Data:** public GEC corpora (Lang-8, NUCLE, FCE, W&I+LOCNESS from BEA-2019). Since real data is limited, pretrain first on **synthetic** pairs made by injecting errors into clean text (drop/swap articles, wrong verb tense, typos, etc.). Then fine-tune on real data, and finish on the cleanest set (W&I).
- **Loss:** token-level cross-entropy with teacher forcing (the decoder sees the gold previous token), label smoothing 0.1.

Optimizer	AdamW, $\beta = (0.9, 0.98)$, weight decay 0.01, grad clipping 1.0
Learning rate	Fine-tuning a pretrained model: 3e-5 to 1e-4, ~1K warmup steps, then linear/cosine decay. From scratch: ~5e-4 with inverse-sqrt schedule.
Batch	~4K to 8K tokens/GPU (dynamic batching by length), mixed precision bf16
Iterations	Synthetic stage ~100K to 200K steps; real-data fine-tune ~10K to 30K steps; pick the checkpoint by dev $F_{0.5}$
Eval	$F_{0.5}$ with ERRANT / M2 scorer (precision weighted higher, since a wrong “fix” is worse than a missed one)

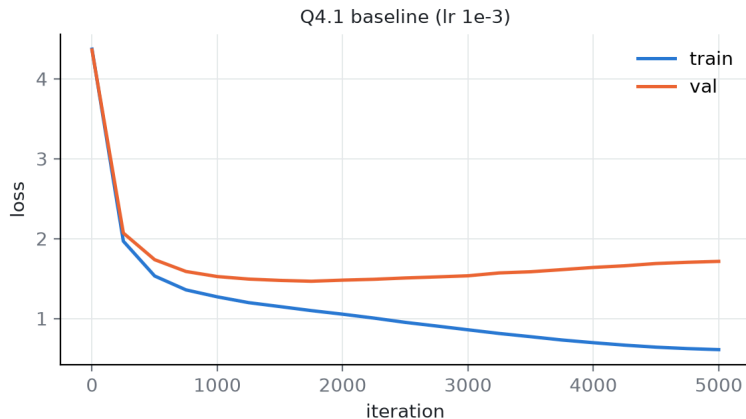
Q4. Mini-LLM (nanoGPT, char-level Shakespeare)

Setup for every run: default config/train_shakespeare_char.py (6 layers, 6 heads, $d = 384$, context 256, batch 64, dropout 0.2, AdamW, lr 1e-3 with cosine decay to 1e-4, 5000 iters). **Seed 1337 and 5000 iterations are fixed for all runs.** Each variant changes exactly one thing. Every variant is a flag in model.py (norm_type, mlp_type, pos_enc, n_kv_head), with one config per run in config/hw1_*.py. I also tuned the LR: every run was repeated at lr = 2e-3 (min_lr 2e-4), and both numbers are reported. All 12 runs were on one RTX 3080 Ti (PyTorch 2.11, CUDA, bf16 autocast, no torch.compile). Logs are in logs/, and plots for both LRs are in plots/.

run	params	best val, lr 1e-3 (iter)	best val, lr 2e-3 (iter)	val @ 5000, 1e-3 / 2e-3
baseline	10.75M	1.4654 (1750)	1.4627 (1750)	1.714 / 1.713
RMSNorm	10.75M	1.4649 (1750)	1.4731 (2000)	1.700 / 1.728
SwiGLU	10.75M	1.4930 (1250)	1.4958 (1750)	1.881 / 1.774
NoPE	10.65M	1.5335 (3000)	1.5275 (3000)	1.567 / 1.581
RoPE	10.65M	1.4674 (1750)	1.4668 (1500)	1.738 / 1.762
GQA	9.86M	1.4696 (2000)	1.4716 (2000)	1.689 / 1.717

How to read this: every model overfits hard. Val loss bottoms out around iter 1750 and then climbs while train loss keeps dropping (1M characters vs. 10M params). nanoGPT only saves a checkpoint when val improves, so **best val** is the number that matters. Val at 5000 mostly measures overfitting. Each setting is one seed, so I treat gaps under about 0.005 as a tie. Going from lr 1e-3 to 2e-3 never changes the ranking.

4.1 Baseline



Best val **1.4654** at iter 1750, which matches the 1.4697 the nanoGPT README reports. After that it's classic overfitting: train goes to 0.61, val climbs back to 1.71.

4.2 LayerNorm

Which one? Pre-LayerNorm. In Block.forward, the norm is applied **inside** the residual branch, before attention and the MLP: $x = x + \text{attn}(\text{ln}_1(x))$, $x = x + \text{mlp}(\text{ln}_2(x))$, plus a final ln_f . (Post-LN would be $x = \text{ln}(x + \text{attn}(x))$.)

Change: swapped every LayerNorm for RMSNorm: $\text{RMSNorm}(x) = x / \sqrt{\text{mean}(x^2) + \varepsilon} \cdot g$. No mean subtraction, no bias. Parameter count is unchanged (the baseline LN already has no bias).



Result: a tie (1.4649 vs 1.4654, and 1.4731 vs 1.4627 at lr 2e-3). The curves basically sit on top of each other. Makes sense: in a pre-LN model, the mean-centering in LayerNorm isn't doing much, and RMSNorm just drops it. RMSNorm is a bit cheaper with no loss in quality, which is why LLaMA-style models use it.

4.3 MLP

Which activation? GELU: $\text{Linear}(d, 4d) \rightarrow \text{GELU} \rightarrow \text{Linear}(4d, d)$, about $8d^2$ params.

Change: SwiGLU: $\text{SwiGLU}(x) = W_{\text{down}}(\text{SiLU}(W_{\text{gate}}x) \odot W_{\text{up}}x)$. That's 3 matrices, so to keep params equal I set the hidden size to $h = 8d/3 = 1024$: $3 \cdot d \cdot h = 8d^2$. The parameter count comes out **exactly** the same as baseline (10.745M).



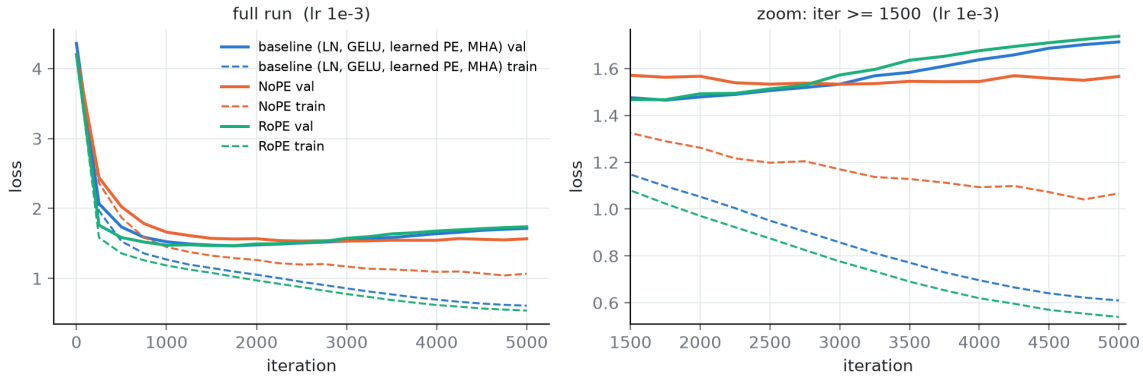
Result: slightly worse here (1.4930 vs 1.4654; 1.4958 vs 1.4627 at 2e-3). SwiGLU actually **fits** faster: lower train loss early, and 0.50 vs 0.61 final train loss at lr 1e-3. But it starts overfitting earlier (best at iter 1250) and ends with the worst val. On this tiny, data-limited problem, extra fitting power just turns into memorization. SwiGLU's usual win shows up in the big-data regime, where you're not overfitting.

4.4 Positional encoding

Which one? Learned absolute position embeddings: `wpe = nn.Embedding(256, 384)`, added to the token embeddings at the input.

Change 1, NoPE: just delete `wpe`. The causal mask still leaks position info (token t can see exactly t tokens), so the model can still work out order implicitly. 98K fewer params.

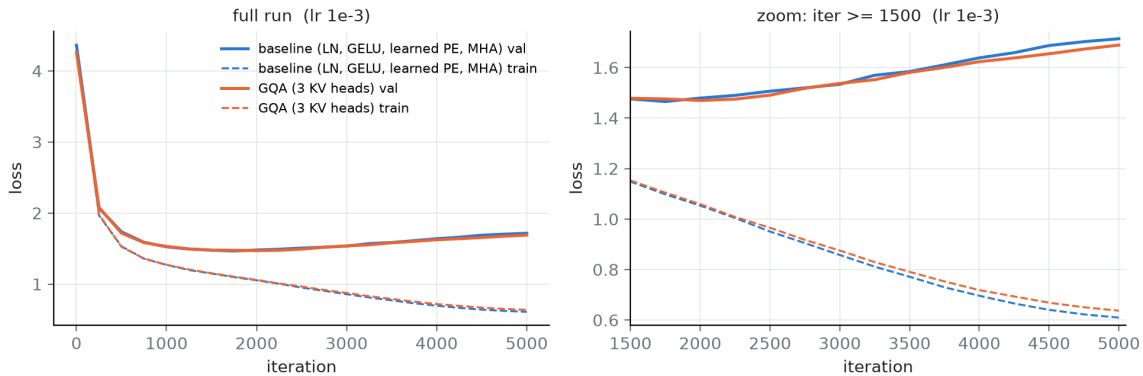
Change 2, RoPE: delete `wpe`, and rotate q and k inside every attention layer (per head, head dim 64, base 10000, consecutive pairs as in Q2). I checked that $q_i^T k_j$ depends only on $i - j$ (same score at (10, 3), (107, 100), (200, 193)).



- **NoPE is clearly worse at its best** (1.5335 vs 1.4654). It learns slower, since it has to infer order from the causal mask alone. It also barely overfits: train only gets to 1.07, and val stays flat around 1.55 to 1.57. So its final-iter val is actually the best of all runs, but that's a side effect of underfitting, not a better model.
- **RoPE ties the baseline at its best** (1.4674 vs 1.4654) and **learns fastest early**: it's ahead of baseline on val for the first 1000 iters. With a 256-token context, there's no length-extrapolation test here, so RoPE's main advantage (relative positions, which generalize better to unseen lengths) doesn't get to show. It also overfits a bit harder by the end.

4.5 GQA

Change: 6 query heads, 3 key/value heads ($n_{kv_head} = 3$). Query heads $2g$ and $2g + 1$ share KV head g (repeat_interleave before attention). The KV projection shrinks from $2d^2$ to d^2 per layer, so the model has **0.88M fewer params** (9.86M vs 10.75M). That's expected; GQA's point is a smaller KV cache at inference.



Result: a tie (1.4696 vs 1.4654; 1.4716 vs 1.4627 at $2e-3$), using 8% fewer parameters. GQA also overfits a little less (val at 5000 is 1.689 vs 1.714). Sharing K/V across query-head pairs costs basically nothing in quality here, and it halves the KV cache, which is the whole point for inference.